Methods I

The A→E generation spine

Generation proceeds through five stages, each implemented as a pure function in its own module (`src/grammar.py`, `src/expand.py`, `src/materialize.py`, `src/verify.py`, `src/sealing.py`). No stage depends on interactive state or network access; every stage takes an immutable input and returns an immutable (or file-system-materialized) output.

```
flowchart LR
    A[Load Grammar] --> B[Expand Spec]
    B --> C[Materialize Child]
    C --> D[Verify Integrity]
    D --> E[Seal + QR]
```

Methods II

Stage A — Grammar loading and validation

`load_grammar(project_root)` reads `manuscript/config.yaml`, extracts the `autopoiesis:` block, and hands it to `parse_grammar()`. Parsing enforces four invariants before a `Grammar` object can exist at all:

1. **The seed must be an integer.** `parse_grammar` reads `block.get("seed", 42)` and raises `GrammarError` if the value is not an `int` — a stray string or float seed in `config.yaml` fails loudly at load time rather than silently coercing.
2. **At least one slot must be defined.** An empty or missing `slots:` list raises `GrammarError("Grammar must define at least one slot")`.
3. **Every slot must have a non-empty name and 1 option.** These checks live in `GrammarSlot.__post_init__`, so they fire the instant a `GrammarSlot` is constructed — a malformed entry cannot survive to become part of a `Grammar`.
4. **No slot may contain duplicate options.** `GrammarSlot.__post_init__` also computes `{o for o in`